

```
1 using System;
2 using System.Globalization;
3 using System.Collections.Generic;
4 using System.Data.Common;
5 using System.Net.NetworkInformation;
6 using System.Reflection.Emit;
7 using System.Security.Cryptography;
8 using System.Windows.Forms;
9 using static System.Windows.Forms.VisualStyles.VisualStyleElement;
10
11 namespace RestaurantSupplier_National
12 {
13     internal class Inventory: Form1
14     {
15
16         public List<Food> list_Dairy = new List<Food>();
17         public List<Food> list_Produce = new List<Food>();
18         private int dairyCounter = 0;
19         private int produceCounter = 0;
20         private Form1 mainform;
21         String answerSymbol;
22         int answerCount;
23
24         public Inventory(Form1 form1)
25         {
26             mainform = form1;
27         }
28
29
30         //Creates Produce objects that are added to general list. Counts how many are created.
31         public void produceListAdditions(String sku, String nam, double cost, int kg)
32         {
33             list_Produce.Add(new Produce(sku, nam, cost, kg));
34             produceCounter++;
35         }
36
37
38
39         //Creates Dairy objects that are added to general list. Counts how many are created.
40         public void dairyListAdditions(String sku, String nam, double cost, int kg)
41         {
42             list_Dairy.Add(new Dairy(sku, nam, cost, kg));
43             dairyCounter++;
44         }
45
```

```
46      //Use this method to get the typed commands from the user and then ↗
      perform actions based upon the input.
47      //This could be a good place to also control how the text appears ↗
      on the labels.
48      public void getCommand() //SC5
49      {
50          String commandAnswer;
51          mainform.lblTotalCounts.Visible = false;
52          mainform.lblEstimatedValue.Visible = false;
53          mainform.lblRetypeCommand.Visible = false;
54
55          String[] commands = { "IMPORT", "DISTRO", "LIST", "END" };
56
57          //Tests that the user entry matches the available commands
58          mainform.lblGreeting.Visible = true;
59          mainform.txtCommand.Visible = true;
60          mainform.txtCommand.Focus();
61          mainform.lblGreeting.Text = "What do you want to do today: ↗
              IMPORT, DISTRO, LIST, or END ? ";
62
63          //Add a check method to see whether the enter has been clicked
64          commandAnswer = mainform.txtCommand.Text;
65          commandAnswer = commandAnswer.ToUpper();
66
67          if (commandAnswer.CompareTo("IMPORT") == 0)
68          {
69              mainform.txtSymbol.Visible = true;
70              mainform.lblFoodCode.Visible = true;
71              mainform.lblFoodCode.Text = "Which food-product do you ↗
                  want to procure?";
72              mainform.txtSymbol.Focus();
73
74          }
75          else if (commandAnswer.CompareTo("DISTRO") == 0)
76          {
77              mainform.txtSymbol.Visible = true;
78              mainform.lblFoodCode.Visible = true;
79              mainform.lblFoodCode.Text = "Which food-product do you ↗
                  want to sell?";
80              mainform.txtSymbol.Focus();
81          }
82          else if (commandAnswer.CompareTo("LIST") == 0)
83          {
84              getInventoryList();
85          }
86          else if (commandAnswer.CompareTo("END") == 0)
87          {
88              MessageBox.Show("Goodbye");
89              mainform.Close();
```

```
90     }
91     else
92     {
93         MessageBox.Show("Command Entry Error");
94         mainform.txtCommand.Text = "";
95     }
96
97 }
98
99 //This will allow the user to type the commands to perform one of
    the actions
100 //The actions are already programmed in advance
101 /**list
102 private double getInventoryTotalValue() //SC8
103 {
104     double tempValue = 0;
105     foreach (Food s in list_Dairy)
106     {
107         tempValue += s.getValue();
108     }
109     foreach (Food s in list_Produce)
110     {
111         tempValue += s.getValue();
112     }
113     return tempValue;
114 }
115
116 }
117 /**list
118 //This is not a required method. It will get all of the print
    information from the food-product objects.
119 public override String ToString()
120 {
121
122     String inventoryPrinter = "";
123     inventoryPrinter = inventoryPrinter + "\n
    =====PRODUCE=====\\n\\n";
124     for (int a = 0; a < list_Produce.Count; a++)
125     {
126         inventoryPrinter = inventoryPrinter + list_Produce
            [a].ToString() + "\\n";
127     }
128     inventoryPrinter = inventoryPrinter + "\n
    =====DAIRY=====\\n\\n";
129     for (int a = 0; a < list_Dairy.Count; a++)
130     {
131         inventoryPrinter = inventoryPrinter + list_Dairy
            [a].ToString() + "\\n";
```

```
133
134     }
135
136     return inventoryPrinter;
137
138 }
139
140
141 //NOTE: all of the label and text box visibility code is optional ➤
142 //This method prints list
143 private void getInventoryList() //SC9
144 {
145     mainform.lblTotalCounts.Visible = true;
146     mainform.lblList.Visible = true;
147     mainform.lblEstimatedValue.Visible = true;
148     mainform.lblTotalCounts.Text = "There are " + produceCounter + ➤
149         " produce food-products and " + dairyCounter + " dairy ➤
150         food-products.";
151
152     mainform.lblList.Text = ToString();
153     mainform.lblEstimatedValue.Text = "This is the estimated value ➤
154         of the inventory for the restaraunt supplier: " + ➤
155         getInventoryTotalValue().ToString("C", ➤
156         CultureInfo.CurrentCulture); //SC9
157
158     mainform.lblFoodCode.Visible = false;
159     mainform.lblKG_QA.Visible = false;
160     mainform.txtSymbol.Visible = false;
161     mainform.txtSymbol.Text = "";
162     mainform.txtKG_QA.Text = "";
163     mainform.txtKG_QA.Visible = false;
164
165     mainform.lblRetypeCommand.Visible = true;
166     mainform.lblRetypeCommand.Text = "Please enter in your next ➤
167         command.";
168     mainform.txtCommand.Text = "";
169 }
170
171 //Non required helper method. This is used to control the farming ➤
172 //process
173 public void procure_InventoryDataEntry()
174 {
175     MessageBox.Show("procure method runs");
176     Boolean verify;
177     answerSymbol = mainform.txtSymbol.Text;
178     answerSymbol = answerSymbol.ToUpper();
```

```
174         verify = supplySKUVerification(answerSymbol);
175         if (verify==false)
176         {
177             MessageBox.Show("You have entered an incorrect symbol");
178             mainform.lblFoodCode.Text = "Incorrect symbol entry. Reenter the correct symbol.";
179             mainform.txtSymbol.Text = "";
180             mainform.txtSymbol.Focus();
181         }
182         else
183         {
184             mainform.lblKG_QA.Visible = true;
185             mainform.txtKG_QA.Visible = true;
186             mainform.txtKG_QA.Focus();
187             mainform.lblKG_QA.Text = "How many kg do you want to procure? (NOTE: 1000 kg is the maximum limit)";
188         }
189     }
190
191     public void procureInventory()
192     {
193         String answer;
194         Boolean verify = false;
195         mainform.txtKG_QA.Focus();
196         try //SC12
197         {
198             answer = mainform.txtKG_QA.Text;
199             answerCount = Convert.ToInt32(answer);
200         }
201         catch(FormatException)
202         {
203             MessageBox.Show("You have entered the wrong data type");
204             mainform.txtKG_QA.Clear();
205             mainform.txtKG_QA.Focus();
206             return; //escape from the method
207         }
208
209         verify = dataEntryVerification(answerCount, answerSymbol, true); // Check if the variables are reaching this point
210
211         if (!verify)
212         {
213             MessageBox.Show("Incorrect Data Entry. Reenter a value greater than 0 and not greater than 1000");
214
215             mainform.txtKG_QA.Clear();
216             mainform.txtKG_QA.Focus();
217
218         }
```

```
219     }
220     else if (verify==true)
221     {
222         exchange(answerSymbol, answerCount, true);
223     }
224     else
225     {
226         mainform.lblKG_QA.Text = "Warning Logic Error: This is the
            final else statement"; //Code should not reach this
            point
227     }
228 }
229
230 //Non required helper method. This is used to control the selling
    process
231 public void distro_InventoryDataEntry()
232 {
233     Boolean verify;
234     answerSymbol = mainform.txtSymbol.Text;
235     answerSymbol = answerSymbol.ToUpper();
236     verify = supplySKUVerification(answerSymbol);
237     if (verify == false)
238     {
239         MessageBox.Show("You have entered an incorrect symbol");
240
241         mainform.txtSymbol.Text = "";
242         mainform.txtSymbol.Focus();
243     }
244     else
245     {
246         mainform.lblKG_QA.Visible = true;
247         mainform.txtKG_QA.Visible = true;
248         mainform.txtKG_QA.Focus();
249         mainform.lblKG_QA.Text = "How many kg do you want to
            sell?";
250     }
251 }
252
253 //called from Form1, used to check data entry. Not required
254 public void distro_InventoryFiles()
255 {
256     String answer;
257     Boolean verify = false;
258     mainform.txtKG_QA.Focus();
259     try //SC10
260     {
261         answer = mainform.txtKG_QA.Text;
262         answerCount = Convert.ToInt32(answer);
263     }
```

```
264         catch (FormatException)
265         {
266             MessageBox.Show("You have entered the wrong data type");
267             mainform.txtKG_QA.Clear();
268             mainform.txtKG_QA.Focus();
269             return; //escape from the method and forces user to have
                to enter in the correct tonnage values
270         }
271
272         mainform.label1.Text = answerCount.ToString();
273         verify = dataEntryVerification(answerCount, answerSymbol,
                false); // Check if the variables are reaching this point
274         if (!verify)
275         {
276             MessageBox.Show("Incorrect Data Entry. Reenter a value
                greater than 0, and less-or-equal to amount in the
                foodStock");
277             mainform.txtKG_QA.Clear();
278             mainform.txtKG_QA.Focus();
279         }
280         else if (verify == true)
281         {
282             exchange(answerSymbol, answerCount, false);
283         }
284         else
285         {
286             mainform.lblKG_QA.Text = "You have entered in the wrong
                value type";
287         }
288     }
289
290     //Use this method to verify if the user is correctly entering
    symbols
291     private Boolean supplySKUVerification(String sym) //SC10
292     {
293
294         for (int i = 0; i < list_Dairy.Count; i++)
295         {
296             if ((sym.Equals(list_Dairy[i].getSKU())))
297                 return true;
298         }
299         for (int i = 0; i < list_Produce.Count; i++)
300         {
301             if ((sym.Equals(list_Produce[i].getSKU())))
302                 return true;
303         }
304         return false;
305     }
306
```

```
307      //Use this method to verify if the user is correctly entering numerical values
308      private Boolean dataEntryVerification(int amt, String sym, Boolean procureOR_Distro) //SC11
309      {
310
311          int temp = 0;
312          if (procureOR_Distro) //true mines
313          {
314              if (amt <= 0 || amt > 1000)
315                  return false;
316              if (procureOR_Distro)
317                  return true;
318          }
319
320          bool tempHold_TF = false;
321
322          for (int i = 0; i < list_Dairy.Count; i++) //generating a list of
323          {
324              if ((sym.CompareTo(list_Dairy[i].getSKU()) == 0))
325              {
326                  temp = list_Dairy[i].getTons();
327                  if (amt > temp)
328                      tempHold_TF = false;
329                  else
330                      tempHold_TF = true;
331              }
332          }
333
334          for (int i = 0; i < list_Produce.Count; i++) //generating a list of
335          {
336              if ((sym.CompareTo(list_Produce[i].getSKU()) == 0))
337              {
338                  temp = list_Produce[i].getTons();
339                  if (amt > temp)
340                      tempHold_TF = false;
341                  else
342                      tempHold_TF = true;
343              }
344          }
345
346          return tempHold_TF;
347      }
348
349      //Not required method. This determines if the user is importing or distributing, and then adds or subtracts via object methods.
350      private void exchange(String sym, int amt, Boolean
```

```
procureOR_Distro) //true mines...false sells
{
351     {
352
353         for (int i = 0; i < list_Dairy.Count; i++) //generating a list of
354             {
355                 if (sym.CompareTo(list_Dairy[i].getSKU()) == 0)
356                 {
357                     if (procureOR_Distro) list_Dairy[i].procureKG(amt);
358                     else { list_Dairy[i].distroKG(amt); }
359                 }
360             }
361         for (int i = 0; i < list_Produce.Count; i++) //generating a list of
362             {
363                 if (sym.CompareTo(list_Produce[i].getSKU()) == 0)
364                 {
365                     if (procureOR_Distro) list_Produce[i].procureKG(amt);
366                     else { list_Produce[i].distroKG(amt); }
367                 }
368             }
369         getInventoryList();
370     }
371 }
372 }
373 }
```